

Feature Engineering Notes

Causal Event-Option Snapshot Construction

Kripta Studios

July 2026

Contents

1	Goal	2
2	Snapshot Time	2
3	Training to Live Equivalence	2
4	Policy Replay	2
5	Candidate Identity	2
6	Minute and Entry Alignment	2
7	Exit Features	3
8	Audits	3
9	Practical Rule	3

1 Goal

Feature engineering must make the training, backtest, replay, and live paths equivalent. A model can only be promoted if the live feed can reproduce the exact feature families used in validation without looking forward.

2 Snapshot Time

For a snapshot at time t , allowed information is:

$$\mathcal{F}_t = \{\text{spot, option chain, OHLC, Greeks, prior completed context observed at or before } t\}.$$

Forbidden information is anything in (t, ∞) , including exit prices, future option highs/lows, future spot returns, and future labels.

3 Training to Live Equivalence

The main files involved are:

```
neural/jepa/build_event_option_dataset.py
neural/jepa/event_option_live_snapshot.py
neural/jepa/event_option_live_scorer.py
neural/jepa/event_option_component_live.py
```

The production contract requires that features exported by the dataset builder can be re-constructed by the live snapshot builder.

4 Policy Replay

Policy replay is a required validation step. It checks that runtime policy logic reproduces historical selected trades without missing or extra keys. A candidate should not be live-ready if the replay cannot explain the trades produced in validation.

5 Candidate Identity

Candidate identity is based on ticker, date, time, expiry mode, side, bucket, and policy family. This identity allows comparison between:

- historical candidate rows;
- runtime replay rows;
- live candidate audit rows;
- paper order intents.

6 Minute and Entry Alignment

Live entries are selected from event-option snapshots generated during the entry window. The bot uses regular trading hours and the policy entry window:

```
10:00-14:30 ET
```

No model should depend on future rows created later in the day.

7 Exit Features

Exit state for an already open live trade may use current premium, current PnL, elapsed hold time, and high water mark observed so far. It may not use the future maximum premium or future close price.

The current live exit is mechanical:

```
stop=-60%
tp=1000%
trail=50%/25%
min_hold=30m
max_hold=180m
```

8 Audits

Run feature leakage and live drift checks when changing feature code:

```
python neural/jepa/audit_event_option_feature_leakage.py ...
python neural/jepa/audit_live_feature_drift.py --day-dir rt_data/YYYYMMDD --strict-features
```

Validation should fail closed when a required live feature is missing.

9 Practical Rule

If a feature is easy to compute in backtest but hard to compute live at the same timestamp, do not use it for production. The live feed is the authority for deployable feature availability.